

ICML 2024 | How a Nonlinear Transformer Learns to Learn in Context

RPI and IBM give the first training-dynamics analysis of a Transformer that keeps both nonlinear attention and a nonlinear MLP — and prove when in-context learning generalizes, in-domain and under distribution shift.

Large language models can do something that still feels a little uncanny: without any retraining or fine-tuning, you drop a handful of input-output examples into the prompt, and the model follows the pattern to solve a brand-new task. This is in-context learning (ICL) — the ability that makes a pretrained model feel plug-and-play, and one of the clearest things that sets modern LLMs apart from classical models.

The awkward part is that we do not really understand it. Why does training a Transformer produce ICL in the first place, and how far does that ability reach — does it work only on tasks resembling those seen in training, or does it hold up when the test distribution shifts? These questions have stayed mostly open, for a concrete mathematical reason: self-attention is nonlinear through its softmax, the MLP is nonlinear through its ReLU, and stacking those two nonlinearities makes the training objective nonconvex, out of reach of the tools that work for linear models.

This ICML 2024 paper, from Rensselaer Polytechnic Institute and IBM's Watson research center, gives the first training-dynamics analysis of a one-layer Transformer that keeps both nonlinear self-attention and a nonlinear MLP. It proves that the trained model generalizes in context in two settings — in-domain and under distribution shift — and it gives the first theoretical account of how pruning affects ICL.

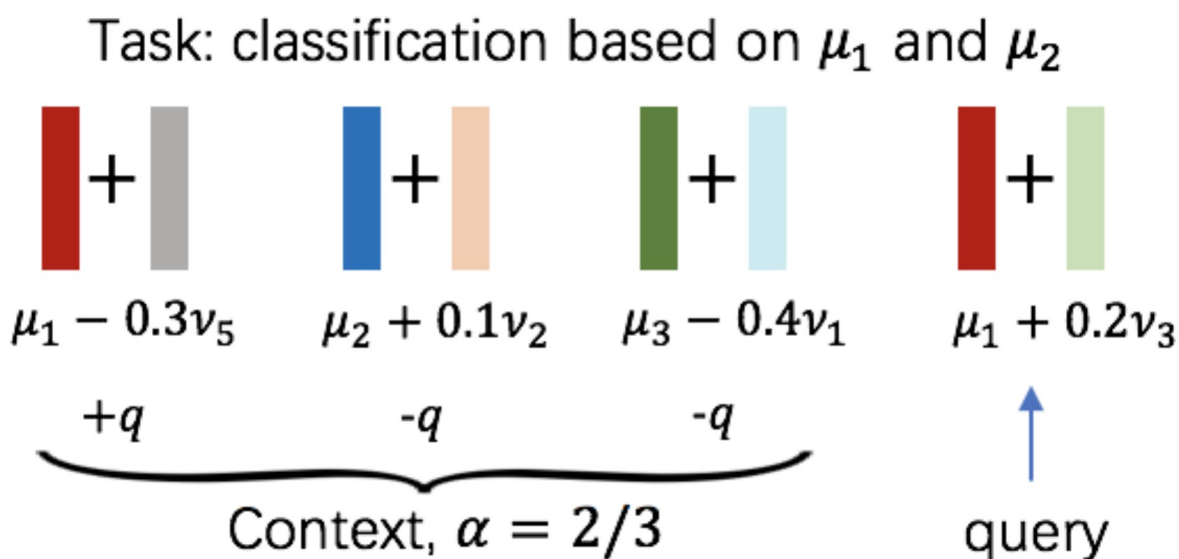


Figure 1. The ICL setup. A prompt is a sequence of labeled context examples plus a query to be predicted. Each input is a label-deciding relevant pattern (the IDR pattern, μ) added to a label-irrelevant pattern v ; here the context length is $l=3$ and the fraction of context examples sharing the query's relevant pattern is $\alpha=2/3$.

For the full details, the paper is at <https://arxiv.org/abs/2402.15607> (arXiv:2402.15607); it is a purely theoretical analysis, backed by verification experiments on synthetic data and a small GPT-2.

Earlier theory dropped the hard part

Prior attempts to explain ICL each kept only half of the picture. Some drop the nonlinear self-attention; others replace the nonlinear MLP with a linear one; most study linear regression rather than classification. As a result, none can characterize how to train a model that still generalizes when the training and test distributions differ, and none analyze how pruning a trained model changes its in-context ability.

That gap matters in practice. Practitioners routinely prune large models to cut inference cost while hoping to keep their in-context skills intact — yet there was no theory saying when pruning is safe for ICL. This paper takes aim squarely at those two blank spots.

A minimal model that stays genuinely nonlinear

To make the training trajectory tractable, the authors shrink the model to its essentials while insisting on two real nonlinearities: a single-head, one-layer self-attention with a softmax, followed by a two-layer ReLU perceptron. Each query is padded with a length- l context of example pairs, embedded into a prompt, and the model is trained by minimizing a hinge loss over a small subset of binary classification tasks — with the full gradient-descent trajectory of the query, key, value, and MLP weights tracked throughout.

The model output is $F(\Psi;P) = a^T \cdot \text{ReLU}(W_O \sum_i W_V p_i \cdot \text{attn}(\Psi;P,i))$, with $\text{attn}(\Psi;P,i) = \text{softmax}((W_K p_i)^T W_Q p_{\text{query}})$. Two quantities run through the whole analysis: α , the fraction of context examples that share the query's relevant pattern, and β , the magnitude of the relevant features. As we'll see, how many iterations training takes and how long a context it needs are governed almost entirely by these two.

How the trained model works: attention focuses, the MLP amplifies

The paper factors the trained model's prediction into two steps. First, attention: the attention layer places its largest weight on the context examples that share the query's relevant pattern (in the schematic, that attention score reaches 0.8). Second, the MLP: the large-magnitude neurons of $W_O W_V$ align with the label direction and amplify the labels of exactly those attended examples, and the prediction follows. Attention finds the right examples; the MLP reads out their answer.

This is not just a story — it shows up in the training curves. The figure below tracks how the magnitude of the attention-relevant weights evolves over training epochs.

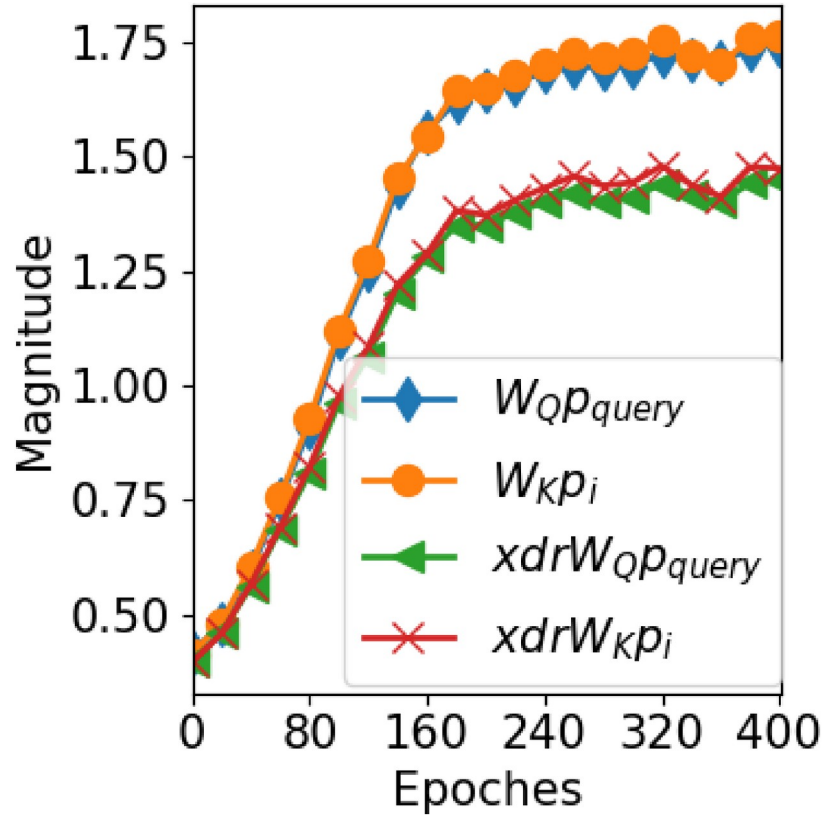


Figure 2. Magnitude of the attention weights over training epochs. The norms of $W_Q p_{query}$ and $W_K p_i$ grow and stabilize within roughly 300 epochs, and are carried almost entirely by the component aligned with the relevant pattern (the xdr curves) — evidence that training steers attention onto the label-deciding relevant patterns.

The main result: generalize in-domain, and under a shift

The main theorem gives a clean scaling: with $T = \Theta(\eta^{-1} M_1 \alpha^{(-2/3)} \beta^{(-2/3)} \sqrt{\log M_1})$ iterations and $N = BT$ samples, the trained one-layer Transformer drives its in-domain generalization error down to $O(\epsilon)$. One detail is especially striking — the model only needs to be trained on an $(M_1 - 1)^{(-1/2)}$ fraction of all in-domain tasks, yet it generalizes to the rest.

The out-of-domain half is the more surprising one. The same trained model also generalizes to tasks whose relevant patterns have changed — provided the new patterns are linear combinations of the training patterns with coefficient sum $S_1 \geq 1$. That condition holds up sharply when checked on a real model.

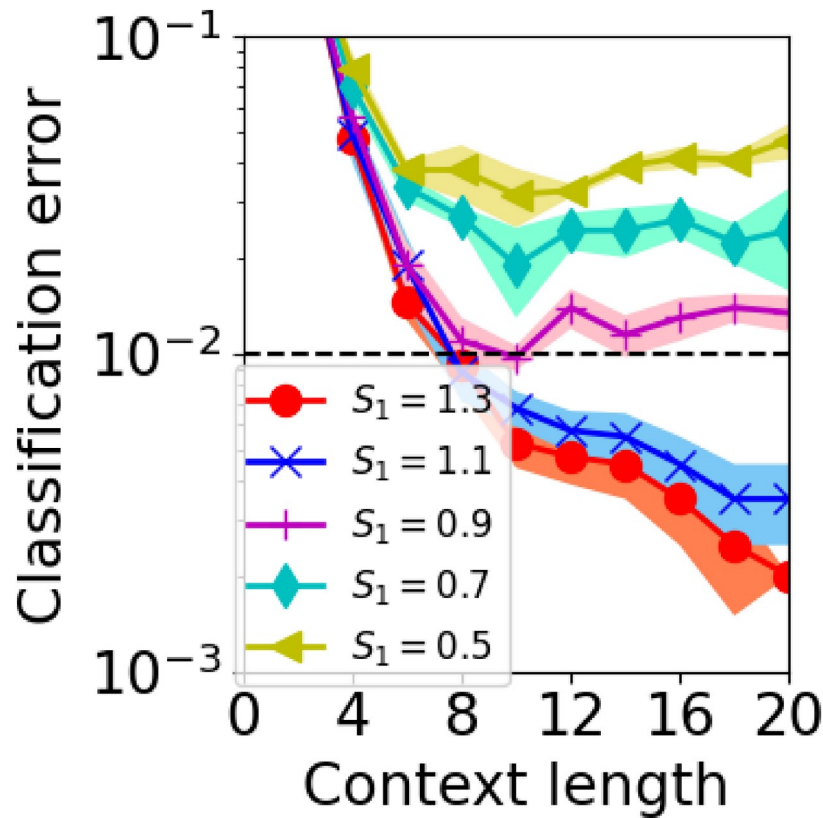


Figure 3. Out-of-domain (OOD) ICL classification error on a small GPT-2 versus context length, grouped by the linear-combination strength S_1 . The $S_1=1.3$ and 1.1 curves fall below 0.01 (dashed line) as the context grows, while $S_1=0.9$, 0.7 , and 0.5 stay above it — matching the theory's $S_1 \geq 1$ condition for OOD generalization almost exactly.

Against classical classifiers, ICL wins where it's hard

The paper also pits ICL against several classical classifiers — logistic regression, SVMs with Gaussian and linear kernels, and 1- and 3-nearest neighbors — using the context examples as their training data. When the relevant fraction is high ($\alpha'=0.8$), the methods are close. But in the harder $\alpha'=0.6$ regime, ICL becomes the most sample-efficient of the group. One reading is that it is better at filtering out irrelevant data and more tolerant of label noise than the classical baselines.

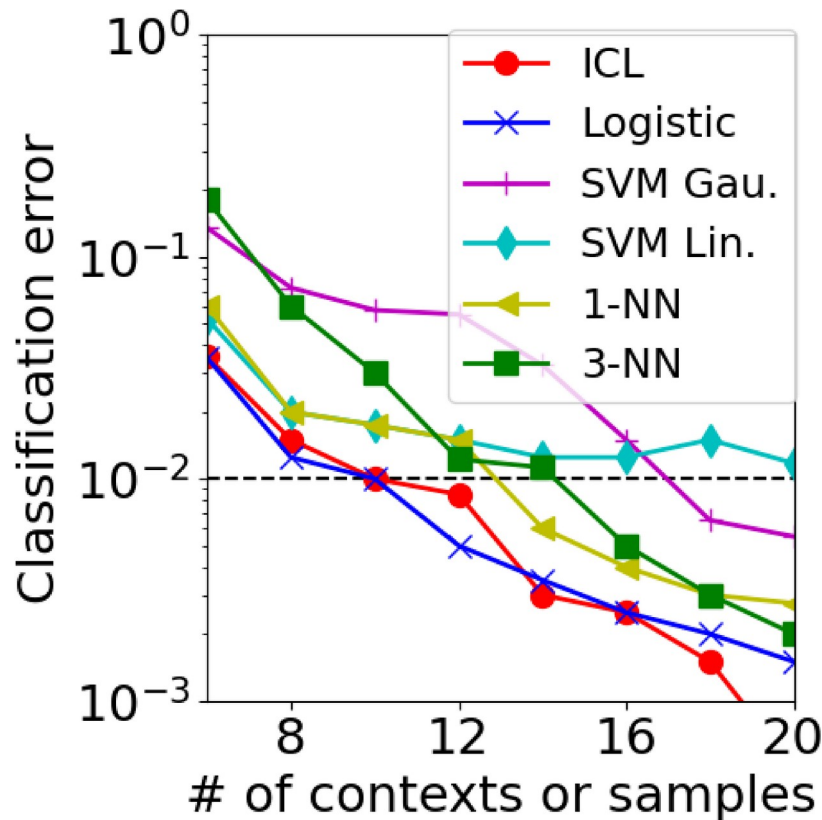


Figure 4. Binary classification error versus the number of contexts/samples at $\alpha'=0.8$, for ICL, logistic regression, SVM (Gaussian and linear kernels), and 1-/3-nearest neighbors. The methods are close here; in the harder $\alpha'=0.6$ setting (in the paper) ICL's sample-efficiency lead widens.

Why magnitude-based pruning preserves ICL

Finally, pruning. The paper proves that removing the small-magnitude neurons of W_O raises the error only to $O(\epsilon + M_1^{(-1/2)})$ — essentially lossless — whereas pruning the large-magnitude neurons raises it at least linearly in the pruning rate. That mirrors the mechanism above: the large-magnitude neurons are the ones carrying the ICL signal, so the small ones are cheap to discard.

The boundary is practically useful: it says that to slim down a model that already does ICL, you should prune by neuron magnitude and keep the pruning rate under this safe line, rather than dropping neurons at random. The theory tells us not just that pruning is possible, but how to prune and how much. The figure below puts magnitude-based and random pruning on the same axes, so we can watch the two diverge as the pruning rate climbs.

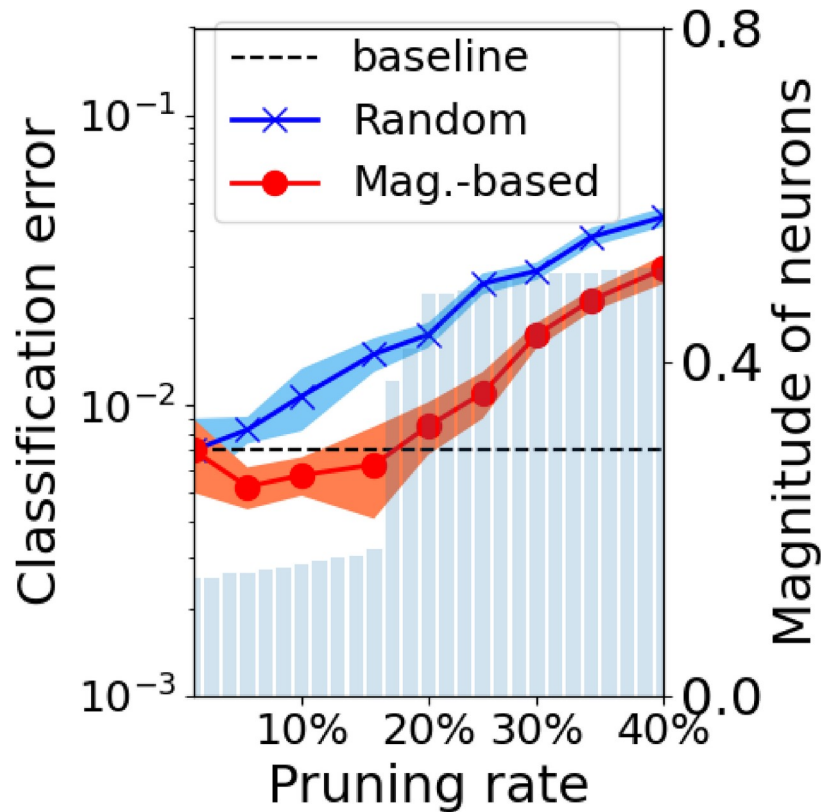


Figure 5. OOD classification error on the one-layer Transformer as the pruning rate grows. Magnitude-based pruning (red) hugs the no-pruning baseline (dashed) up to roughly 15% to 20%, while random pruning (blue) degrades quickly. The light-blue bars are each neuron’s magnitude, confirming that the low-magnitude neurons can be removed nearly for free.

Takeaway and boundaries

The message is that in-context learning in a nonlinear Transformer is not a black box. A one-layer model that keeps both nonlinear attention and a nonlinear MLP can be provably trained to generalize in context — in-domain and under distribution shift — with the training cost set mainly by how much of the context shares the query’s relevant pattern. The mechanism is attention concentrating on the matching examples while the ReLU MLP amplifies their labels, which is also why the small-magnitude neurons, carrying little of that signal, can be pruned almost for free: a principled reason magnitude-based pruning preserves ICL.

The boundaries are worth stating plainly. The analysis rests on a one-layer, single-head Transformer and synthetic data ($M_1=6$ relevant patterns, $M_2=24$ irrelevant ones), and the out-of-domain guarantee depends on the linear-combination, coefficient-sum ≥ 1 condition. Whether real multi-layer LLMs follow the same laws exactly remains open — but the paper’s experiments on a small GPT-2 give the theory early, consistent support.

The scaling at a glance

Table 1. How training cost and generalization scale (α : relevant fraction in the training context; α' : relevant fraction at test; M_1 : number of relevant patterns; S_1 : linear-combination strength).

Quantity	Scaling / condition
Iterations T, samples N	$\propto \alpha^{(-2/3)}$
Required context length (train)	$\propto \alpha^{-1}$
Required context length (test)	$\propto \alpha'^{-1}$
Fraction of tasks needed for training	only $(M_1-1)^{(-1/2)}$ of in-domain tasks
OOD generalization condition	error below 0.01 once $S_1 \geq 1$
Near-lossless magnitude-pruning rate	about 15% of W_O neurons

Put the pieces together and the paper draws a fairly clear theoretical line under why in-context learning works and how far you can economize on it: attention picks out the relevant examples, the ReLU MLP amplifies their labels, the training budget scales as $\alpha^{(-2/3)}$ in the relevant fraction α , and pruning should track neuron magnitude. For engineers compressing large models, that is at least a theory-backed intuition to work from.

There is, of course, real distance between a one-layer Transformer on synthetic data and a production multi-layer LLM, and the paper does not pretend otherwise. But it takes an ability that had lived mostly at the level of an empirical phenomenon and, for the first time, grounds it in provable training dynamics — which is reason enough to want the same analysis pushed toward deeper, more practical models.